

Etiquetado con modelo no supervisado

Diseño e Implementación de una Red Social Distribuida

Victor Hugo Pacheco Fonseca C411

Jose Agustin del Toro C412

Asignatura: Machine Learning

2025

Índice

1. Resumen	2
2. Metodología de Etiquetado de Datos	2
2.1. Dataset Inicial	2
3. Ingeniería de Características para Isolation Forest	2
4. Implementación y Optimización de Isolation Forest	3
4.1. Configuración del Modelo	3
4.2. Experimentación con Parámetro Contamination	3
4.3. Análisis de los Resultados	3
5. Verificación Manual de Resultados	4
6. Creación de Reglas de Detección	4
7. Conclusiones	4

1. Resumen

Se implementó un sistema híbrido de detección de anomalías en logs web Apache combinando técnicas de machine learning no supervisado (Isolation Forest) con un sistema de reglas heurísticas. El proceso incluyó etiquetado semiautomático de datos, ingeniería de características, optimización de hiperparámetros, validación manual de resultados y enriquecimiento iterativo del sistema de reglas.

2. Metodología de Etiquetado de Datos

2.1. Dataset Inicial

- **Origen:** Logs de acceso Apache de servidores web productivos
- **Período:** Noviembre 2025
- **Tamaño:** ~1,000,000 registros
- **Estado:** Aproximadamente 50 % pre-etiquetado mediante sistema de reglas existente

3. Ingeniería de Características para Isolation Forest

- **A. Características Numéricas (StandardScaler):**
 - **Frecuencias y ocurrencias:**
 - `uri_occurrence`: Frecuencia absoluta de cada URL
 - `user_agent_occurrence`: Frecuencia de cada User-Agent
 - `ip_occurrence`: Frecuencia por dirección IP
 - `ioc_occurrence`: Conteo de Indicators of Compromise
 - **Características de longitud:**
 - `uri_length`: Longitud del string URI
 - `user_agent_length`: Longitud del User-Agent
 - `uri_decoded_length`: Longitud después de URL decode
 - **Estadísticas de respuesta:**
 - `response_size`: Tamaño de respuesta en byte
- **B. Características Categóricas (OneHotEncoding):**
 - `method_GET`, `method_POST`, `method_HEAD`
 - `protocol_HTTP_1.1`, `protocol_HTTP_2.0`
 - `status_2xx`, `status_4xx`, `status_5xx`

4. Implementación y Optimización de Isolation Forest

4.1. Configuración del Modelo

```
IsolationForest(
n_estimators=100,
max_samples=0.25,      # 25% para reducir swamping/masking
max_features=0.5,      # 50% de características por árbol
bootstrap=False,
n_jobs=-1,             # Paralelización completa
random_state=42        # Reproducibilidad
)
```

4.2. Experimentación con Parámetro Contamination

Se evaluaron 10 valores de `contamination` en un dataset de 576,767 registros con 84,530 anomalías reales (14.65 %):

Cuadro 1: Resultados de experimentación con diferentes valores de `contamination`

Contamination	Accuracy	Precision	Recall	F1-Score	TP	FP	FN
0.001 (0.1 %)	0.8544	0.9705	0.0066	0.0132	560	17	83,970
0.005 (0.5 %)	0.8548	0.6397	0.0218	0.0422	1,845	1,039	82,685
0.01 (1 %)	0.8568	0.6657	0.0454	0.0850	3,839	1,928	80,691
0.02 (2 %)	0.8588	0.6351	0.0867	0.1525	7,327	4,209	77,203
0.03 (3 %)	0.8564	0.5486	0.1123	0.1864	9,492	7,811	75,038
0.05 (5 %)	0.8617	0.5823	0.1986	0.2962	16,786	12,040	67,744
0.075 (7.5 %)	0.8474	0.4599	0.2353	0.3113	19,893	23,364	64,637
0.1 (10 %)	0.8512	0.4879	0.3079	0.3775	26,026	27,319	58,504
0.15 (15 %)	0.8718	0.5612	0.5743	0.5677	48,549	37,965	35,981
0.2 (20 %)	0.8548	0.5033	0.6865	0.5808	58,026	57,256	26,504

4.3. Análisis de los Resultados

■ Alto Precision, Bajo Recall (0.001-0.01):

- Contamination 0.001: Precision 97.05 % pero Recall solo 0.66 %
- Detecta pocas anomalías pero con alta confianza
- Ideal para entornos donde falsos positivos son críticos

■ Zona de Balance (0.03-0.1):

- Contamination 0.03: F1-Score 0.1864, Precision 54.86 %, Recall 11.23 %
- Contamination 0.05: Mejor equilibrio con F1-Score 0.2962
- Recall mejora significativamente mientras Precision disminuye gradualmente

■ Alto Recall, Bajo Precision (0.15-0.2):

- Contamination 0.15: Recall 57.43 %, Precision 56.12 %, F1-Score 0.5677
- Contamination 0.2: Recall 68.65 % (más alto) pero Precision baja a 50.33 %
- Detecta más anomalías pero genera más falsos positivos

5. Verificación Manual de Resultados

Una vez obtenido el puntaje de anomalía del modelo, se ordenaron los registros según su *anomaly score*. Se revisaron manualmente los casos con valores más bajos (más anómalos) para determinar:

- Si realmente correspondían a ataques.
- Si existía un patrón repetido que justificara crear una regla específica.

Este proceso permitió:

- Confirmar ataques reales detectados por el modelo.
- Identificar falsos positivos.
- Descubrir nuevas familias de ataques no contempladas inicialmente.

6. Creación de Reglas de Detección

Cuando se identificaba un ataque real con un patrón claro, se diseñaba una regla para detectarlo automáticamente. Ejemplos de patrones incorporados:

- Inyecciones SQL con operadores como `CAST`, `CASE WHEN`, `||`, `::text`.
- URLs con comentarios ofuscados (`/* ... */`).
- Terminaciones típicas de SQLi (`--+`).
- Bots maliciosos con *User-Agent* falsificado.

Cada regla se añadía al sistema para mejorar la detección futura y reducir la dependencia del análisis manual.

7. Conclusiones

El proceso permitió:

- Mejorar la calidad del etiquetado del dataset.
- Detectar ataques complejos que no estaban cubiertos por reglas previas.
- Ajustar el modelo de *Isolation Forest* para maximizar su utilidad.
- Crear reglas específicas que fortalecen la seguridad del sistema.

El enfoque híbrido (modelo + verificación manual + reglas) demostró ser efectivo para la detección progresiva de nuevas amenazas.